

**COVERING CODES CHECKED BY THE LEAN KERNEL:
 $K_7(9, 4) \leq 1134$, TEN MORE UPPER BOUNDS, $K_2(6, 1) = 12$,
AND $K_7(4, 2) = 19$**

THIAGO PATZDORF

ABSTRACT. We prove in Lean 4, against Mathlib v4.34.1 and checked by the kernel with no `sorry` and no `native_decide`, upper bounds on the covering number $K_q(n, R)$ for eleven cells, each by an explicit code smaller than the corresponding entry in the tables we consulted. The largest improvement is $K_7(9, 4) \leq 1134$, against a best published bound of 1475; three more cells unchanged in the tables since 2011 come down to $K_7(10, 4) \leq 5607$ (from 6517), $K_5(11, 4) \leq 2875$ (from 3125) and $K_5(10, 5) \leq 162$ (from 175); the others are $K_7(8, 3) \leq 1887$, $K_5(10, 4) \leq 625$, $K_5(9, 3) \leq 1250$, $K_5(7, 2) \leq 500$, $K_5(9, 4) \leq 250$, $K_4(10, 4) \leq 192$ and $K_5(9, 5) \leq 50$. The codes are unions of cosets of a linear code plus a small patch, and we give two certificates: one walks digit prefixes and costs 12 CPU-hours for $(\mathbb{Z}/7\mathbb{Z})^9$; the other works on syndromes and costs minutes (the 5616-word code: 11 minutes on four cores). The base of the $K_7(9, 4)$ code was chosen from an enumeration of all 6362 equivalence classes of non-degenerate $[9, 3]_7$ codes. We also prove $K_2(6, 1) \geq 11$ by a short double-counting argument and the classical value $K_2(6, 1) = 12$ by a search whose soundness is a theorem. Finally we determine $K_7(4, 2) = 19$ (previously $17 \leq K_7(4, 2) \leq 19$), and the equality is a theorem checked by the kernel with no hypothesis: the upper bound is an explicit code, and the non-existence of an 18-word code reduces, by lemmas proved in Lean, to 70 SAT instances whose LRAT refutations are replayed by a verified checker inside the kernel (284 modules, 37.8 CPU-hours); the same refutations were also checked by two independent checkers outside Lean and confirmed by a second, independent encoding. We also describe a machine-checked ledger of covering-code upper bounds, with formally certified exact entries: 642 of the 1145 upper bounds in the tables are theorems of the Lean kernel (551 formalized, 91 of them also checked by a second, executed program), almost all of them from a single theorem, a generic bound $K_q(n, R) \leq |C|$, construction rules, explicit witnesses and codes transcribed from the literature and rebuilt here; the exact value is a theorem on both sides in 2 cells, $K_2(6, 1)$ and $K_7(4, 2)$.

Version 0.9 adds three exact values that are, as far as we can tell, potentially new: we did not find them in the literature we searched. Their lower bounds are *not* theorems of the kernel: each rests on a reduction proved on paper and on computational certificates checked by exact programs outside Lean. First, $K_3(6, 2) = 17$ (previously $15 \leq K_3(6, 2) \leq 17$, the lower bound by Bertolo, Östergård and Weakley [1] and the upper bound by Hämäläinen and Rankinen [8]): after a normalization that fixes a minimal fibre, every hypothetical code with 15 or 16 words falls into one of 12049 or 12674 instances, whose linear relaxations are infeasible by 12054 and 13099 integer Farkas certificates checked in exact arithmetic; a second pipeline sharing no code with the first (another reduction, enumeration and canonical form, with VeriPB and LRAT proofs) reproduces both steps independently. Second, we generalize the fibre lemma behind $K_7(4, 2) = 19$ to $K_q(n, n - 2)$ and obtain $K_7(6, 4) = 14$ (previously $13 \leq K_7(6, 4) \leq 15$ [9, 11]): a new 14-word code, whose covering property is formalized in Lean (kernel), and LRAT refutations of the 8008 fibre profiles of a 13-word code. Third, $K_7(5, 3) = 17$ (previously $15 \leq K_7(5, 3) \leq 17$, both bounds announced in Kéri's tables [10]), by LRAT refutations of one profile for 15 words and of all 201376 profiles for 16 words, two of them split into 1812 and 4953 cubes; version 0.9.1 adds a 17-word code of our own whose covering property is formalized in Lean (kernel), so both bounds of this value are now established here, the upper one as a theorem of the kernel. Each of the three lower bounds survived an adversarial review; the gaps that remain are stated in the text.

This draft (7 October 2026) also states the method as a pipeline, witness $\rightarrow$ independent verifier $\rightarrow$ Lean kernel $\rightarrow$ objective bound, in which search programs, solvers and language models only propose and never decide; describes the infrastructure that runs it (an official exact verifier, the Lean targets, and a batch executor for ephemeral spot machines whose budget is enforced in code); and places the project among the 2026 results on mathematics with artificial intelligence, none of which, in the sources we read, concerns covering codes. Every count of the ledger quoted in this note is generated from the ledger by a script, and a test fails if the text diverges from it.

1. INTRODUCTION

The covering number $K_q(n, R)$ is the least cardinality of a code $C \subseteq A^n$, with $|A| = q$, such that every word of A^n lies at Hamming distance at most R from some word of C [5]. Upper bounds come from constructions and lower bounds from counting; the tables of Kéri [10] and the recent work of Gijswijt and Polak [7] and of Marosi [13] record the current state.

Upper bounds are certificates: a list of words and the claim that it covers. Checking the claim is a finite computation, but for q^n in the tens of millions it is done by programs that are rarely audited. Florath [6] built a Lean library of covering-code bounds whose derivations are replayed by the kernel; it formalizes the sphere-covering bound and records many cells. This note adds certificates that the kernel checks for eleven cells where the codes are new, a certificate that scales by working on syndromes, formal determinations of a small classical value and of a value that was open, and a ledger in which every upper bound we could certify is a theorem. Version 0.9 adds three exact values whose lower bounds are of a different nature (Sections 7 and 8), and records where the same methods stop (Section 9); version 0.9.1 adds a kernel proof of $K_7(5, 3) \leq 17$ by a code of our own, so the two bounds of $K_7(5, 3) = 17$ no longer depend on an announced, unpublished code. The method common to all sections is stated in Section 14, the infrastructure in Section 15, and related work on mathematics with artificial intelligence in Section 19.

Throughout, we distinguish three kinds of statement. A *theorem of the kernel* (or *formalized statement*) is the name of a declaration that Lean’s kernel accepted with no hypothesis. A *verified computational certificate* is a reduction proved on paper together with certificates (LRAT refutations, integer Farkas multipliers, or VeriPB proofs) that an exact program outside Lean checked; the solvers that produced the certificates need not be trusted, but the reduction, the encoder and the checker do. We call such a result *independently reproduced* when a second pipeline, sharing no code with the first, reaches the same conclusion, and we say how much of it was reproduced. An *observation* is a measurement that supports no claim and enters no proof. Every statement labelled *Theorem*, and every upper bound of Table 1, is of the first kind; the lower bounds of Sections 7 and 8 are of the second kind and are labelled *computational results*.

The proofs use Lean 4 [12] and Mathlib [14]. The source is <https://github.com/thiagopatzdorf/Mathematica>, tag v0.9.1. A statement below is a theorem of the kernel only when it is the name of a declaration in that tree; the computational lower bounds of Sections 7 and 8 are claims of the second kind and name no declaration. The default target `CoveringLean` builds in minutes; the prefix certificates form the target `CoveringHeavy`, the syndrome certificates the target `CoveringSyn`, the refutations for $K_7(4, 2)$ and its final theorem the target `CoveringK742Sat`, and the certified ledger the target `CoveringLedger`.

2. DEFINITIONS

In the development `CoveringA2` a word is a function $\text{Fin } n \rightarrow \mathbb{Z} \bmod q$, distance is Mathlib’s `hammingDist`, and

$$\text{Covers } R \ C : \iff \forall x, \exists c \in C, d(x, c) \leq R.$$

In `CoveringA6` the alphabet is $\text{Fin } q$, with the same distance and the same notion of covering, and

$$\text{IsK } q \ n \ R \ k : \iff (\exists C, |C| = k \wedge \text{Covers } R \ C) \wedge \forall C, |C| < k \rightarrow \neg \text{Covers } R \ C,$$

which is the equality $K_q(n, R) = k$.

3. UPPER BOUNDS FOR $K_7(9, 4)$

Theorem 1. *There is $C \subseteq (\mathbb{Z}/7\mathbb{Z})^9$ with $|C| = 1134$ that covers at radius 4. In Lean: `Syn.K7_9_4_le_1134_syn`, stated as*

$$\exists C : \text{Finset } (\text{Fin } 9 \rightarrow \mathbb{Z} \bmod 7), \quad C.\text{card} = 1134 \wedge \text{Covers } 4 \ C.$$

The library proves the same statement for five codes of this cell, found in succession: 1351 words (`CoveringKernel.K7_9_4_le_1351_kernel`, and again as `Syn.K7_9_4_le_1351_syn`), 1285 words (`Syn.K7_9_4_le_1285_syn`), 1141 words (`Syn.K7_9_4_le_1141_syn`), 1137 words (`Syn.K7_9_4_le_1137_syn`) and 1134 words. Each code is a list of natural numbers $\sum_i w_i 7^i$ in strictly increasing order, and the same list is a text file in `data/codes/`; the canonical SHA-256 of the 1134-word code begins `a48e3a42`.

Structure. All five codes are three cosets $r_j + C_0$ of a linear $[9, 3]_7$ code C_0 (1029 words) together with a patch. Write C_0 as the kernel of $H = [I_6 \mid A]$; the three cosets cover exactly the points whose syndrome is within distance-4 reach of one of three syndromes, and the remaining syndromes are *orphans*, whose 343 points each must be covered by the patch. For the 1351-word code there are 27 orphans and 322 patch words; for 1285, 24 orphans and 256 patch words; for 1141, with A given by the rows 666, 065, 652, 643, 621, 615 and coset syndromes 0, 7708, 4191, there are only 6 orphans (2058 points) and 112 patch words; the 1137- and 1134-word codes keep the same base and need 108 and 105 patch words.

First certificate: digit prefixes. A Boolean function `go` walks the coordinates one at a time, keeping for each prefix the codewords still within reach and their remaining budget of mismatches; `go_sound` proves that `go` returning `true` implies that every word has a codeword at distance at most R . `covers_of_go` turns that into `Covers`, using that Mathlib’s distance agrees with the digit-wise count, and `cert_of_go` adds the cardinality, from a kernel check that every entry is below q^n and the list is strictly increasing. `go_split` fixes a prefix of coordinates in advance, so the computation becomes many leaf theorems, each closed by `decide +kernel`; for $K_7(9, 4) \leq 1351$ there are $7^4 = 2401$ leaves in 95 files, about 12 CPU-hours in total. The pipeline accepts the binary Hamming code of length 7 and rejects a copy of it with one word replaced; both are tests in the library.

Second certificate: syndromes. The second certificate uses the structure. `SynCheck` (core Lean only, no Mathlib) defines Boolean checks over a transversal of the q^{n-k} syndrome classes: each class gets a witness (coset, message) or is marked orphan; each orphan’s q^k points are checked against the list; and each coset word $r_j + mG$ is shown to be in the list. `SynBridge` (with Mathlib, of fixed size) proves the checks sound, transports coverage along the translation by C_0 , and concludes $\exists C, C.\text{card} = M \wedge \text{Covers } R \ C$ in the theorem `syn_cert`. The leaves are `decide +kernel` over blocks of 4096 classes and import only `SynCheck`. The 1351-word code costs about 8 CPU-minutes this way instead of 12 CPU-hours, and each leaf stays below 1.8 GB. Each certificate also contains an `example` in which a wrong witness is rejected by the kernel.

Remark. The best upper bound we found is $K_7(9, 4) \leq 1475$, by Marosi [13] (arXiv version 3, September 2026); Kéri’s tables [10] give 1843. The best lower bound we know is 264. That the bound is new is our reading of those sources on 3 October 2026, not something the kernel checks.

4. A LOWER BOUND WITHOUT SEARCH: $K_2(6, 1) \geq 11$

Lemma 2. *For distinct $a, c \in (\mathbb{F}_2)^6$, the balls of radius 1 about a and c meet in 0 or 2 words.*

Proof. At distance 1 the intersection is $\{a, c\}$; at distance 2 it is the two words between them; at distance at least 3 it is empty. In Lean the case $a = 0$ is a kernel decision over 64 words (`even_inter_zero`), and translation gives the general case (`inter_card_translate`). $\square$

Theorem 3. *Every code that covers $(\mathbb{F}_2)^6$ at radius 1 has at least 11 words (`CoveringA6.K_2_6_1_ge_11`).*

Proof. Let $K = |C|$ and let $m(y)$ be the number of codewords whose ball contains y , so $m(y) \geq 1$ and $\sum_y m(y) = 7K$. For each x put $S(x) = \sum_{y \in B(x)} m(y) = \sum_{c \in C} |B(x) \cap B(c)|$. Then $S(x) \geq 7$, and if $x \notin C$ every term is even by Lemma 2, so $S(x) \geq 8$. Summing $S(x) + [x \in C] \geq 8$ over all 64 words and using $\sum_x S(x) = 7 \sum_y m(y) = 49K$ gives $50K \geq 512$, hence $K \geq 11$. The counting is `excess_bound`, stated for any family of symmetric balls of size 7 with even pairwise intersections. $\square$

This replaces a hypothesis that the previous version of this note left open: the statement `no_cover_2_6_1_le10_uncond` is the earlier conditional one with the condition removed.

5. AN EXACT VALUE: $K_2(6, 1) = 12$

Theorem 4. *SC.K_2_6_1_eq12 : IsK 2 6 1 12.*

The value is classical [18]. The upper bound is an explicit 12-word code (`code12`, encodings 0, 1, 2, 15, 28, 23, 57, 58, 59, 39, 44, 52). For the lower bound, a search `chkN` on 64-bit masks takes a state (words still allowed, covered set, forbidden centres), picks the least uncovered word, and branches on the centres that could cover it, forbidding at each branch the siblings already tried. It prunes when the remaining words cannot cover what is left, at 7 words per centre. `chkN_sound` proves that a `true` answer means no completion exists. A translation argument lets one assume the word 63 is in the code. The search from that root is cut at depth 2 into 38 states, each refuted by `decide +kernel` in its own file. The Lean database of Florath [6] records $10 \leq K_2(6, 1) \leq 12$ at this cell.

6. AN EXACT VALUE: $K_7(4, 2) = 19$

Kéri's tables give $17 \leq K_7(4, 2) \leq 19$. The upper bound is the partition construction (the word 0000 together with an MDS $[4, 2, 3]_3$ code on each of the alphabets $\{1, 2, 3\}$ and $\{4, 5, 6\}$); in Lean it is `K742.K_7_4_2_1e_19`, decided by the kernel over the 2401 words. For the lower bound, two words of $\mathbb{Z}_7^4$ are at distance at most 2 exactly when they agree in at least two coordinates. A counting argument on a box of shortened words (Lemma 0) gives $K_6(3, 1) \geq 18$ and $K_7(3, 1) \geq 21$; with it, a fibre $\{c \in C : c_j = a\}$ of a code of radius 2 with at most 18 words has at least 2 words (Lemma 1, `K742.fiber_card_ge_two`). The multiset of fibre sizes per coordinate, up to permuting coordinates, is one of 70 *profiles* for $M = 18$ (`profiles18_card`), and each profile gives a CNF that is satisfiable whenever a code of that profile exists. Following the method Florath used for $K_8(4, 2) = 23$ [6], we refuted all 70 CNFs with CaDiCaL and LRAT proofs (3.0 GB in total), each checked by `lrat-check` and by an independent checker; the 15 profiles of $M = 17$ are refuted the same way. An independent encoding (one variable per word, coverage by radius-2 balls, a totalizer for the cardinality and only lexicographic symmetry breaking) refutes again all 15 profiles of $M = 17$ and all 70 of $M = 18$, with another solver; an adversarial review checked the lemmas, the list of profiles and the symmetry breaking, including by verifying that 1200 randomly relabelled codes satisfy the CNF of their profile and that four deliberately wrong symmetry-breaking constraints are caught. Together this gives $K_7(4, 2) = 19$, and the whole argument is now a theorem of the kernel.

Theorem 5. *K742.K_7_4_2_isK : IsK 7 4 2 19, and K742.K_7_4_2_eq_19 states the same equality; neither has a hypothesis.*

The proof composes three kernel-checked pieces. The first is the reduction: Lemmas 0 and 1 and the profile lemmas above, plus the bridge `K742Ponte.ponte18`, which renames coordinates and symbols so that every 18-word code of radius 2 satisfies the CNF of one of the 70 profiles; the bridge is stated for the CNF without the symmetry-breaking clauses of the first encoding, so the normal form used to break symmetry did not have to be formalized (dropping them costs 7 times more solver time but only 11% more hints after trimming). The second is an LRAT checker written for the kernel (`LratK`, Lean core only), with a soundness proof (`LratK.unsat_of_good`); the CNF is generated inside Lean and the kernel checks, for every refutation, that it equals the DIMACS file that was refuted. The third is the 70 refutations themselves (`K742Sat.refut18`): CaDiCaL proofs without symmetry breaking, trimmed by `lrat-trim` to 5.6 million steps and 227 million hints, split into 284 modules that the kernel checked in 37.8 CPU-hours on two virtual machines, with at most 9.2 GB per process. Mathlib's `FromLRAT`, the mechanism Florath used for $K_8(4, 2)$, ran out of memory (killed above 12 GB) on the proof of profile 64, 5 MB after trimming, which is why we wrote our own checker. The default target contains the reduction and the bridge; the refutations and Theorem 5 form the target `CoveringK742Sat`, outside continuous integration because of its

cost, and the per-module record of that build (time, memory, exit code) and the `#print axioms` output are in the repository.

7. $K_3(6, 2) = 17$ BY LINEAR-PROGRAMMING CERTIFICATES

Kéri's tables give $15 \leq K_3(6, 2) \leq 17$: the lower bound is due to Bertolo, Östergård and Weakley [1], improving 14 by Blass and Litsyn [4]; the upper bound is a 17-word code of Hämäläinen and Rankinen [8]. The semidefinite bound of Gijswijt and Polak [7] gives 13.12 at this cell. The case of 16 words was attacked in 2026 by Raval [17], who excludes 6 of 38 normalized branches, and by Florath [6], whose notes record the attempt as a failure; neither claims a global bound. We show that no code with 15 or with 16 words exists. The upper bound $K_3(6, 2) \leq 17$ is a theorem of the kernel in the ledger of Section 13 (`CoveringLedger.K3_6_2_1e_17`, from an explicit 17-word witness); nothing else in this section is checked by the kernel, and in particular $K_3(6, 2) \geq 17$ is not formalized in Lean.

The reduction by a minimal slice. Let $C \subseteq \mathbb{Z}_q^n$ cover at radius R with $M \leq q^n$ words; repeated words can be replaced by words outside C , so the words may be assumed distinct. The fibre $F(j, a) = \{c \in C : c_j = a\}$; let s^* be the least $|F(j, a)|$, so $s^* \leq \lfloor M/q \rfloor$. By Hamming isometries we move a minimal fibre to coordinate 0 and symbol 0, order the other symbols of coordinate 0 so that the block sizes $t_b = |F(0, b)|$ satisfy $t_1 \geq \dots \geq t_{q-1} \geq s^*$, and move the projection $K \subseteq \mathbb{Z}_q^{n-1}$ of $F(0, 0)$, a set of s^* distinct points, to the canonical representative of its class under $S_q \wr S_{n-1}$.

Lemma 6 (slice filter). *Let U be the set of $y \in \mathbb{Z}_q^{n-1}$ at distance more than R from every point of K . Then $|U| \leq (M - s^*) V_q(n-1, R-1)$.*

Proof. A point $(0, y)$ with $y \in U$ is not covered by $F(0, 0)$, so it is covered by a word c with $c_0 \neq 0$, which spends one difference on coordinate 0 and is therefore within distance $R-1$ of y on the other coordinates. Each of the $M - s^*$ words outside $F(0, 0)$ covers at most $V_q(n-1, R-1)$ such points. $\square$

An *instance* is a triple (s^*, K, t) with K canonical and passing the filter, and t a non-increasing block vector of sum $M - s^*$. It is a 0–1 system in one variable z_c per point $c \in \mathbb{Z}_q^n$ with five families of constraints: coverage ($\sum_{d(c,x) \leq R} z_c \geq 1$ for every x), size ($\sum z_c = M$), the slice ($z_c = 1$ on $\{0\} \times K$ and $z_c = 0$ on the other points with $c_0 = 0$), blocks ($\sum_{c_0=b} z_c = t_b$) and fibres ($\sum_{c_j=a} z_c \geq s^*$ for $j \geq 1$, valid because s^* is the minimum). For $K_3(6, 2)$ there are 729 variables; the filter reads $|U| \leq 11(M - s^*)$ and excludes $s^* \leq 1$. For $M = 15$ there are 12 049 instances (5, 108, 936 and 11 000 for $s^* = 2, 3, 4, 5$); for $M = 16$ there are 12 674 (18, 124, 1461 and 11 071).

Completeness.

Lemma 7. *Every code of M distinct words and radius R , once normalized, satisfies all constraints of an instance in the list.*

Proof. The normalization uses only isometries, so it preserves coverage and fibre sizes; the constraints hold by construction. It remains that the canonical K is in the list. The enumeration does not start from sets: it runs over multisets of columns written as restricted growth strings and then takes a canonical form, the minimum over orderings of the points. Let S be any set of s distinct points, read in some order as an $s \times (n-1)$ matrix. Renaming the symbols of each column by order of first appearance is a symbol permutation, so each column becomes a restricted growth string; sorting the columns is a coordinate permutation. The result is one of the enumerated multisets, isometric to S , and its points are still distinct. The filter is applied before the canonical form, which loses nothing because $|U|$ is invariant under isometry, and the canonical form is an isometry invariant, so every class that passes the filter appears exactly once. $\square$

This step, that every set appears as a multiset of restricted growth columns before the canonical form, was missing from the first written version and was supplied after the adversarial review below. As an independent check of the count, Burnside’s lemma for s -subsets of $\mathbb{Z}_3^5$ under $S_3 \wr S_5$ gives 1, 1, 5, 35, 490 and 11 075 orbits for $s = 0, \dots, 5$, which equals the size of the unfiltered enumeration with pairwise distinct forms; with $|U|$ recomputed by brute force, exactly the listed configurations pass the filter for $M = 15$ and for $M = 16$. The SHA-256 of the two instance lists begin **5a07459e** ($M = 15$) and **3cb5b5cc** ($M = 16$), and the lists were regenerated from scratch with the same digests.

Integer Farkas certificates. For each instance we solve the dual of the linear relaxation, with multipliers $y \geq 0$ on the inequalities and μ free on the equalities, and round to integers. With $g = y^\top G + \mu^\top E$, the certificate is accepted when

$$y \cdot h + \mu \cdot e > \sum_c \max(g_c \ell_c, g_c u_c),$$

where $\ell_c \leq z_c \leq u_c$ are the bounds after the slice is fixed; this excludes even fractional solutions. When the root does not suffice we branch on single variables $z_c \in \{0, 1\}$ and certify every leaf. For $M = 15$, 12 047 instances close at the root and two branch (with 5 and 2 leaves): 12 054 certificates. For $M = 16$, 12 593 close at the root and 81 branch (from 2 to 33 leaves, at most 23 distinct variables): 13 099 certificates. Each certificate uses only the five families above; in particular none of the auxiliary lemmas we used to understand the problem (the linear-programming form of Lemma 6, the balanced-profile identity, aggregated linear programs) enters it. Generation took about 20 and 29 minutes on two processes, with HiGHS.

Computational result 8. There is no code in $\mathbb{Z}_3^6$ with 15 or 16 words and covering radius 2. Together with the 17-word code of [8], $K_3(6, 2) = 17$. To our knowledge both lower bounds 16 and 17 are new (Section 18).

The state of Result 8 is a verified computational certificate, independently reproduced for both values of M (see below); it is not a theorem of the kernel.

Exact verification. The checker rebuilds every system from (s^*, K, t) alone, never from a stored matrix, works with Python integers and the standard library only, requires y to be a non-negative integer and μ an integer, uses the strict inequality, checks that the leaves of each instance form a complete branching tree, and checks that the records are exactly the instances of the list, in order, against the SHA-256 of the list. It accepts all certificates in 7 seconds for each value of M . The certificates and the instance lists are in the repository, in `tools/exatos/k362/contagem/dados/` (files `K3_6_2_M15_*` and `K3_6_2_M16_*`), with the checker `verificar.py` in the parent directory.

Adversarial reviews. Two reviews tried to break the claim, one for each value of M . They rechecked the completeness proof line by line against the code (finding the missing step above), ran the Burnside count, regenerated the lists, and checked all certificates again with a minimal Farkas checker written from scratch in exact rationals, which shares no code with the generator or with the first checker; for $M = 16$ that checker had to be rewritten first, because its tree-completeness test enumerated 2^v assignments and the deepest tree has 23 variables. Mutated certificates (a negative multiplier, coverage or fibre rows removed, a certificate moved to another instance, a tree with a leaf removed) were rejected by both checkers. As a guard against a false proof, 26 known 17-word codes, under random isometries, fall into instances whose relaxation is feasible and for which the generator produces no certificate, and 31 224 certificates for $M = 15$ transplanted to those feasible instances were all rejected; 5776 sets of 15 points that pass the filter (and, for $M = 16$, 442 subcodes of 16 words and 3069 sets of 16 points) all land on listed instances. Both reviews conclude that the claim survives, with no gap of high severity; their remaining reservations were independence (a

single enumeration and normalization) and the absence of a formal proof. The first is answered by the reproduction below; the second remains.

Independent reproduction. A second pipeline was written from the idea of the minimal fibre alone, without importing or copying any code of the first. Its reduction does not enumerate the block vector t (only $\sum_{c_0 \neq 0} z_c \leq M - s^*$), so each class of K is one instance: 11 496 instances for $M = 15$ (1, 27, 468 and 11 000 for $s^* = 2, 3, 4, 5$) and 11 592 for $M = 16$ (3, 31, 487 and 11 071), the same classes as above. The classes are enumerated as multisets of restricted-growth columns with canonical forms over row orderings, checked against Burnside’s lemma and against a canonical form computed by nauty on a graph of its own; the same code reproduces the published classifications of the optimal codes for $K_3(5, 2) = 8$ (one class) and $K_3(6, 3) = 6$ (28 classes). Each instance is first refuted on a projected relaxation of 243 variables, and otherwise on the full 0–1 system with the fibre constraints. For $M = 15$ all 11 496 instances are refuted by RoundingSat with VeriPB proofs, all checked by VeriPB, and 11 171 of them also by CaDiCaL with LRAT proofs checked by `lrat-check`. For $M = 16$, 11 582 instances are refuted with checked VeriPB proofs; the other 10, whose proofs exceeded 3 GB, are split by the exact fibre sizes of the first coordinates into 106 leaves, 101 refuted with checked VeriPB proofs and 5 by Farkas certificates checked in exact rationals by a checker written from scratch. No satisfiable formulation and no code with 15 or 16 words appeared. This reproduces Result 8 with another reduction, another enumeration and canonical form, other proof systems and other checkers; it shares the idea of the minimal fibre and, for 5 leaves of $M = 16$, the type of certificate.

A ladder of readable lemmas. The Farkas certificates are weight vectors over 729 points without a human reading. As an observation, which is not a new proof, we classified each instance by the most readable step that kills it, each step with a small certificate checked in integers: a packing in U of more than $M - s^*$ points pairwise at distance at least 3 (pigeonhole), a weighted version of Lemma 6, the same with the fibres, and three steps that also use the complement of the slice and the blocks. For $M = 15$ the pigeonhole step kills 5668 of the 12 049 instances and the weighted slice 5952 more; 416 need the fibres, 11 the complement, and 2 need branching. For $M = 16$ the counts are 1271, 8988, 1891 and 432; 11 instances are killed only by the linear program of the whole instance and 81 only by branching. The instances whose whole linear relaxation is feasible are exactly those whose Farkas certificate has more than one leaf (2 for $M = 15$, 81 for $M = 16$), so for them the next readable step would have to be an integrality argument.

An observation from the SAT side. Before the linear-programming certificates, the same instances for $M = 15$ were attacked with a pseudo-Boolean solver (RoundingSat) with VeriPB proofs; six instances, all with $s^* = 5$, took from 3.5 to more than 30 minutes against a median of 3.6 seconds. An autopsy found that they are not hard: each is refuted in at most 1.4 seconds by a literal subset of its own constraints that only looks at the slice $c_0 = 0$, with VeriPB proofs, and the same two subsets refute 12 040 of the 12 049 instances. What is hard for the solver is finding that refutation inside the whole formula, which happens when Lemma 6 holds with little slack; symmetry of the formula and near-feasibility were measured and ruled out as explanations. This is an observation and is not used in Result 8.

8. THE FIBRE LEMMA FOR $K_q(n, n - 2)$: $K_7(6, 4) = 14$ AND $K_7(5, 3)$

The proof of $K_7(4, 2) = 19$ in Section 6 reduces to fibre profiles. Here the reduction is stated for every (q, n) with $R = n - 2$; the lower bounds obtained are computational results in the sense of the introduction, not theorems of the kernel, while the new upper bound $K_7(6, 4) \leq 14$ and the upper bound $K_7(5, 3) \leq 17$, by a code of our own, are formalized in Lean (kernel).

Lemma 9 (fibres). *Let $C \subseteq \mathbb{Z}_q^n$ cover at radius $R \leq n - 2$ with M words, and let $F = F(j, a)$ have $s < q$ words. Then $K_{q-s}(n - 1, R - 1) \leq M - s$.*

Proof. For $i \neq j$ let $A_i = \{c_i : c \in F\}$, so $|A_i| \leq s$, and choose $S_i \subseteq \mathbb{Z}_q \setminus A_i$ with $|S_i| = q - s$. Let $B = \{x : x_j = a, x_i \in S_i \text{ for } i \neq j\}$. A word of F differs from $x \in B$ in all $n - 1 > R$ other coordinates, so x is covered by some $c \notin F$, and then $d(x, c) = 1 + d'(x, c) \leq R$, where d' is the distance on the coordinates other than j . Fix $\sigma_i \in S_i$ and let φ_i be the identity on S_i and σ_i elsewhere; φ never decreases agreement with a point of B . So the images under φ of the words outside F , restricted to the coordinates other than j , cover $\prod_i S_i \cong \mathbb{Z}_{q-s}^{n-1}$ at radius $R - 1$ with at most $M - s$ words. $\square$

For $n = 4$, $R = 2$ this is Lemma 1 of Section 6. Let $s_{\min}$ be the least s that the lemma does not exclude; if $q s_{\min} > M$ the lemma alone proves $K_q(n, R) > M$. For $R = n - 2$, a word covers x exactly when they agree in at least two coordinates, so coverage is expressed through the projections of C on pairs of coordinates, one clause of width $\binom{n}{2}$ per point. The *type* of a coordinate is the non-increasing vector of its q fibre sizes, and a *profile* is the multiset of the n types; each profile gives a CNF that is satisfiable whenever a code of that profile exists, with symmetry breaking by the steps (a)–(f) of the $K_7(4, 2)$ encoding, a lexicographic order (g) on consecutive columns of the same type, and the order (h) below.

Lemma 10 (order of equal blocks). *In the normal form one may require, in addition to (a)–(g): (h) for consecutive blocks $b, b + 1$ of coordinate 0 of the same size, the (non-decreasing) sequence of coordinate 1 in block b is lexicographically at most that of block $b + 1$.*

Proof. Symbols of coordinate 0 with fibres of the same size can be permuted, which permutes the blocks of a size group and changes the first-appearance relabelling of coordinate 1. Order each group greedily: at each step choose the pair (block, labelling admissible for the relabelling rule) whose sorted vector of coordinate-1 labels is least. The admissible labellings at step t are the bijections between the new symbols of each class c and the m_c first free labels $F_t \cap c$; the least one gives the smallest label to the most frequent symbol. Let $\min_t(Y)$ be the least vector of block Y at step t and v_t the minimum over remaining blocks. Monotonicity: let Y be chosen at step $t + 1$ with labelling L' . The symbols of Y that were new at step t received distinct labels of $F_t \cap c$; sort them by L' and give them, in that order, the m_c first labels of $F_t \cap c$. This is a labelling L admissible at step t with $L(s) \leq L'(s)$ symbol by symbol, a pointwise inequality that passes to sorted vectors with multiplicities. Hence $v_{t+1} = \text{vec}(Y, L') \geq \text{vec}(Y, L) \geq \min_t(Y) \geq v_t$. The labels given are those of the relabelling rule, sorting within a block is (d), and (f), (g) are redone without touching coordinates 0 and 1. $\square$

The first written proof of Lemma 10 compared with a fixed labelling instead of the minimum over admissible labellings, and is false when the new symbols of a block have different multiplicities; the canonizer that implemented it produced normal forms violating (h) in 10^{-5} to 10^{-3} of random codes, with an explicit counterexample in $\mathbb{Z}_4^3$. Both were found by an adversarial review and corrected as above. The CNF encodes the *statement* of (h) directly, not the canonizer, so the certificates were not affected; the review also found no code lost by the symmetry breaking in 2476 SAT tests over whole orbits, without any canonizer, including the 656 codes on which the old canonizer failed. The corrected proof above is the one proposed by that review and adopted in the repository together with a corrected canonizer, which violated nothing in 1.72 million random codes; a test reproduces the counterexample and fails with the old canonizer. The later review of $K_7(5, 3)$ (below) found every planted mutant of (f) and of (h) caught, but no further review was devoted to the proof of Lemma 10 itself.

$K_7(6, 4) = 14$. The code

000000	011111	100011	122200	212222	221122	333333
343444	434455	444366	555534	565643	656665	666556

covers $\mathbb{Z}_7^6$ at radius 4, as checked by the official checker of the repository and by two independent programs (canonical SHA-256 beginning `9f2c5351`); the best published upper bound was 15, from the alphabet-partition construction of Kéri and Östergård [11], and to our knowledge the code is new. It came from the first profile of the search for $M = 14$ (every symbol twice in every coordinate) and follows the pattern of a 12-word code for $K_6(5, 3)$. The upper bound $K_7(6, 4) \leq 14$ is formalized in Lean (kernel), as the declaration `CoveringK764.K_7_6_4_le_14`, whose axioms are `propext`, `Classical.choice` and `Quot.sound`, by a kernel check of the 14-word list over the $7^6 = 117\,649$ words. For $M = 13$, Lemma 9 with $K_7(5, 3) \geq 14$ excludes $s = 0$ and leaves $s_{\min} = 1$, 11 types and $8008 = \binom{16}{6}$ profiles. All 8008 CNFs were refuted by CaDiCaL with LRAT proofs, all checked by `lrat-check`; a sample was checked again by a second checker.

Computational result 11. No code in $\mathbb{Z}_7^6$ with 13 words has covering radius 4; hence $K_7(6, 4) = 14$.

The adversarial review regenerated 401 CNFs with the same SHA-256, regenerated 27 proofs bit for bit and checked them with `lrat-check` and with `cake_lpr`, a checker extracted from CakeML; enumerated the profiles independently (8008, none missing); and refuted 16 of the 8008 profiles with an independent encoding (one-hot coordinates, exact fibres by totalizers, only trivial symmetry breaking) and another solver, with no disagreement, while the 4 profiles of largest symmetry stayed undecided after 1200 seconds. The state of Result 11 is a verified computational certificate: all 8008 refutations are checked by `lrat-check`, and the review confirmed them on a sample. It is not independently reproduced as a whole: the independent encoding covers 16 of the 8008 profiles (0.2%; the rest would cost at least 200 CPU-hours), and `cake_lpr` was run on 27 of the 8008 proofs. The previous lower bound, 13, comes from the recursion $K_{q+1}(n+1, R+1) \geq \min\{2(q+1), K_q(n, R)+1\}$ of Haas, Schlage-Puchta and Quistorff [9, Thm. 6] with $K_6(5, 3) = 12$; that recursion can never give more than $2(q+1) = 14$ at this cell, and 14 is exactly the value we prove. The lower bound uses only $K_7(5, 3) \geq 14$, below the published 15 [10]; we do not chain it on Result 12, which would make it depend on (h) twice.

$K_7(5, 3)$. The previous bounds are 15, announced by Haas, Quistorff and Schlage-Puchta in Kéri's tables (update of 6 October 2009), and 17, announced by Rivas Soriano (update of 25 January 2006) [10]; we prove $K_7(5, 3) = 17$, and to our knowledge both lower bounds 16 and 17 are new. We did not find a published 17-word code, so until version 0.9 the upper bound was the announced one and was not checked here. Version 0.9.1 closes this gap with a code of our own, found by simulated annealing: the code

```
00534 01114 02654 13005 16302 25243 25426 30611 31551
32131 43365 46062 54226 54443 60150 61630 62510
```

covers $\mathbb{Z}_7^5$ at radius 3, as checked by the official checker of the repository (canonical SHA-256 beginning `d4b4c2ac`). The upper bound $K_7(5, 3) \leq 17$ is formalized in Lean (kernel), as the declaration `CoveringK753.K_7_5_3_le_17`, whose axioms are `propext`, `Classical.choice` and `Quot.sound`, by a kernel check of the 17-word list over the $7^5 = 16\,807$ words (the digit-prefix certificate `go`, closed by `decide +kernel`). For $M = 15$, $s = 0$ would need $K_7(4, 2) \leq 15$ and $s = 1$ would need $K_6(4, 2) \leq 14$; both are false ($K_7(4, 2) \geq 17$ is in Kéri's tables and $K_7(4, 2) = 19$ is Theorem 5; $K_6(4, 2) = 15$ is in Kéri's tables, and we refuted 14 again with 8855 LRAT-checked profiles). So $s_{\min} = 2$, and $7 \cdot 2 = 14 \leq 15$ leaves a single profile, type 3222222 in every coordinate. Its CNF was refuted by CaDiCaL in 221 seconds with an LRAT proof of 1.56 GB, checked by `lrat-check` and by a second checker, and again by `kissat`; the review regenerated the proof bit for bit and checked it with `cake_lpr`. Without (h) `kissat` needed 10 365 seconds, and without (g) and (h) it did not finish in 3 hours.

Computational result 12. No code in $\mathbb{Z}_7^5$ with 15 words has covering radius 3; hence $K_7(5, 3) \geq 16$.

For $M = 16$, $s = 0$ is excluded by $K_7(4, 2) \geq 17$ (Theorem 5 gives 19) and $s = 1$ is not, so $s_{\min} = 1$; there are 28 types and $201\,376 = \binom{32}{5}$ profiles. All of them were refuted by CaDiCaL with LRAT proofs checked by `lrat-check`: 201 374 as whole profiles and two, $4222222 \cdot 3322222^4$ and 3322222^5 , which did not finish in 4 hours, split into 1812 and 4953 cubes on the coordinate 1 of the first 16 words. The run took 722 699 seconds of solver time and 116 197 seconds of `lrat-check`, on three virtual machines; the 4.7 TB of proofs were checked and discarded, and their SHA-256 are kept in the repository (`tools/exatos/fibras/certificados/`), with a script that recounts the closed profiles and regenerates CNFs against the recorded digests. A code with fewer than 16 words can be completed with repeated words, which the encoding allows, so this run alone gives the lower bound 17.

Computational result 13. No code in $\mathbb{Z}_7^5$ with 16 words has covering radius 3; with the 17-word code above, $K_7(5, 3) = 17$.

An adversarial review of $M = 16$ enumerated the 201 376 profiles with a program of its own (none missing, none extra, none duplicated) and regenerated 2000 CNFs with the recorded SHA-256. It proved the completeness of the cube splitting of the three expensive profiles by SAT, without trusting any enumerator: a subset of the clauses of the profile plus one blocking clause per cube is unsatisfiable, with LRAT proofs checked by `lrat-check` and `cake_lpr`, and removing one blocking clause makes it satisfiable inside the freed cube; it checked that every cube is closed. It regenerated 51 records bit for bit (CNF and proof) and checked them with both checkers, and refuted 8 cheap profiles with the independent encoding. Its stated gaps are three: 9 large proofs of its sample (0.73 to 27.8 GB) were not regenerated for lack of disk, so for them the only check is `lrat-check` in the original run; the independent encoding covers only 8 cheap profiles; and the upper bound 17 was taken from the literature, a gap that the 17-word code above, formalized in version 0.9.1, closes. The state of Results 12 and 13 is a verified computational certificate, not independently reproduced as a whole.

9. WHERE THE SAME METHODS STOP

We record the negative results of version 0.9, all with no change to any bound; none is a claim.

Minimal slice and linear programming, ternary. On the open ternary cells with $n \leq 8$ the method of Section 7 does not reach a new bound. For $K_3(7, 3)$ (11–12) the list for $M = 11$ has 143 instances and none dies at the root; for $K_3(6, 1)$, $K_3(7, 2)$, $K_3(7, 1)$ and $K_3(8, 3)$ the list of slices cannot be enumerated with our canonical forms (from about $10^{5.5}$ to 10^{72} orbits before the filter), and random samples are uninformative, since the relaxation also kills slices for values of M above known codes. On known values the method reproduces, with exact certificates, $K_3(5, 2) \geq 8$ and $K_3(5, 1) \geq 27$ (343 instances).

The same, binary. It reproduces with exact certificates the known values $K_2(6, 1) = 12$, $K_2(7, 2) = 7$, $K_2(9, 3) = 7$ and $K_2(8, 2) = 12$ (4, 16, 41 and 1723 instances), and 13 020 of the 13 022 instances of $K_2(10, 3) \geq 12$. On the open binary cells it stops either because the relaxation becomes weak (for $K_2(12, 4)$, 29.5% of 73 029 instances die at the root) or because the list does not fit when s^* is large ($K_2(11, 3)$, $K_2(10, 2)$).

The family $K_q(n, n - 2)$. The 14-word code of $K_7(6, 4)$ is a union of words pure in two blocks of the alphabet, of sizes 3 and 4, neither of which covers all pairs of coordinates; this generalizes the alphabet-partition construction, and we have an exact criterion for such block constructions. Within this construction (an observation, since the unsatisfiable answers have no checked proof) there is no 15-word code for $K_8(7, 5)$ among the forms tried and no 16-word code for $K_7(5, 3)$, consistent with Result 13; it gives a 16-word code for $K_8(7, 5)$ that is not the partition but only equals the published bound. For $K_q(4, 2)$, $16 \leq q \leq 20$, the fibre lemma leaves from 10^{10} to 10^{20} profiles and is out of reach; the block construction reduces those cells to one quantity, whose decisive instances were undecided after 12 000 seconds.

10. THE SPHERE-COVERING BOUND

The library also proves the sphere-covering bound $q^n \leq |C| \cdot V$ (`CoveringA2.sphere_covering`), its strict form when $V \nmid q^n$, and eight numerical instances `CoveringChain.SPH_K*_lb`, among them $K_7(9, 4) \geq 221$. This was the content of version 0.2 of this note. The bound had already been formalized by Florath [6], and all eight instances are below the best lower bounds in the literature (for $K_7(9, 4)$, 264), so they are not claimed as results.

11. TEN MORE UPPER BOUNDS

The same certificates prove ten more upper bounds (Table 1). Each declaration is stated as $\exists C, C.\text{card} = M \wedge \text{Covers } R \ C$ over $\text{Fin } n \rightarrow \text{ZMod } q$, and for each cell the list in Lean decodes to the code file in `data/codes/` with the same canonical SHA-256. For $q = 4$ our literature search was weaker than for $q \geq 5$, and $K_5(10, 4) \leq 625$ is a linear $[10, 4]_5$ code, which should still be compared with the tables of Davydov, Marcugini and Pambianco on linear covering codes.

cell	ours	previous	source	certificate
$K_7(9, 4)$	1134	1475	Marosi	syndromes (also 1137, 1141, 1285; 1351 by prefixes)
$K_7(10, 4)$	5607	6517	Kéri	syndromes (also 5616)
$K_5(11, 4)$	2875	3125	Kéri	syndromes
$K_5(10, 5)$	162	175	Kéri	syndromes
$K_7(8, 3)$	1887	2337	Kéri	syndromes (1893 by prefixes)
$K_5(10, 4)$	625	875	Kéri	prefixes
$K_5(9, 3)$	1250	1275	Kéri	prefixes
$K_5(7, 2)$	500	525	Kéri	prefixes
$K_5(9, 4)$	250	255	Kéri	prefixes
$K_4(10, 4)$	192	208	Kéri	prefixes
$K_5(9, 5)$	50	55	Kéri	prefixes

TABLE 1. Upper bounds proved in Lean and checked by the kernel.

The three new cells of Table 1 (`Syn.K7_10_4_1e_5607_syn`, `Syn.K5_11_4_1e_2875_syn`, `Syn.K5_10_5_1e_162_syn`) have stood in Kéri’s tables since at least 2004; Kéri’s 2011 update and the captures we compared leave them unchanged, and we found no later publication of a smaller code (Section 18). The best lower bounds we know are 1007, 546 [7] and 41. Their canonical SHA-256 begin 7670a7bd, 2a700e2f and 74176af6.

12. SUMMARY OF EXACT VALUES AND LOWER BOUNDS

Table 2 lists the cells where this note changes or certifies a lower bound, with the kind of evidence for each side in the vocabulary of the introduction.

13. A CERTIFIED LEDGER

A machine-checked ledger of covering-code upper bounds, with formally certified exact entries. The repository keeps one record for each of the 1145 cells of Kéri’s tables ($2 \leq q \leq 21$), with the best published lower and upper bounds, the frozen version of the source each comes from, and a certification state for each side: *claimed* (only in a published source), *witness checked* (a certificate checked by an exact program outside Lean), *formalized* (a theorem checked by the kernel with exactly this bound and no hypothesis) and *independently reproduced* (formalized and also checked by a second, executed program). The states are computed by a script from the Lean sources and the certificates, never written by hand.

cell	before	now	upper bound	lower bound
$K_2(6, 1)$	12	12	kernel (explicit code)	kernel (Theorem 4)
$K_7(4, 2)$	17–19	19	kernel (explicit code)	kernel (Theorem 5)
$K_3(6, 2)$	15–17	17	kernel (ledger, witness of [8])	integer Farkas certificates, exact checker (Result 8); independently reproduced
$K_7(6, 4)$	13–15	14	kernel (new code)	LRAT, 8008 profiles (Result 11); reproduced on a sample
$K_7(5, 3)$	15–17	17	kernel (our code, version 0.9.1)	LRAT, 201 376 profiles (Result 13); reproduced on a sample

TABLE 2. Exact values and lower bounds. “Kernel” means a theorem of the Lean kernel with no hypothesis; every lower bound of the last three rows is a verified computational certificate, checked by exact programs outside Lean.

For the upper bounds the library has a generic statement, `CoveringUB.K_le_card`: if C covers at radius R then $K_q(n, R) \leq |C|$, with $K_q(n, R)$ defined as an infimum; and the classical construction rules [5, Ch. 3] as theorems for the same types: direct sum, free and dummy lengthening, puncturing, monotonicity in the radius and projection of the alphabet. A generator picks, for each cell, the cheapest certificate that reaches the best known upper bound (a trivial code, a small explicit witness checked by the kernel, or a rule applied to other cells) and writes one theorem per cell; all of them are collected in the single theorem `CoveringLedger.todas_as_cotas` (target `CoveringLedger`). The trivial codes and witnesses are joined by 13 systematic linear codes, for the entries that Kéri’s tables derive from a Hamming code, a perfect code or a linear code rather than from a code file: the Hamming codes $[15, 11]_2$, $[31, 26]_2$, $[13, 10]_3$, $[6, 4]_5$ and $[8, 6]_7$, the Golay codes $[23, 12]_2$ and $[11, 6]_3$, and six codes found by local search ($[6, 3]_7$, $[7, 2]_7$, $[7, 4]_{11}$, $[8, 5]_{13}$, $[8, 4]_{13}$ with covering radius $n - k - 1$, and $[19, 6]_2$ with radius 5). For them `Syn.lin_cert` gives $K_q(n, R) \leq q^k$ over $\mathbb{Z}/q$ for any q : the identity block makes the encoding injective, and the kernel checks a witness for each of the q^{n-k} points of a transversal, never the list of q^k words; the same covering is checked again by a syndrome evaluator in Python. Version 0.9.1 had 547 certified upper bounds. The ledger now also certifies codes transcribed from four papers (Östergård–Kaikkonen 1998, Östergård–Weakley 1999, Hämäläinen–Honkala–Kaikkonen–Litsyn 1993 and Kéri–Östergård 2005 [11]), each rebuilt and checked by an exact evaluator of our own before it enters, and Corollary 3 of [11] is a generic theorem of the kernel (`CoveringSurj.cobre`) with a Boolean check of surjectivity. The transcription is not trusted: two codes printed in those papers, as transcribed from the page, do not have the stated radius (the $(19, 320)$ code of radius 4 in Table 1 of Östergård–Kaikkonen has radius 5, and the $[19, 6]$ code of their Example 5 has radius 6), and no single-digit correction repairs them; neither enters the ledger. Counting also the theorems of the previous sections, 642 of the 1145 upper bounds are formalized or independently reproduced (551 formalized and 91 independently reproduced).

This is not a verification of the whole table. The other 503 upper bounds are only claimed, and the lower bounds are, with 2 exceptions, inherited from the literature: only $K_2(6, 1) \geq 12$ and $K_7(4, 2) \geq 19$ are theorems here, so $K_2(6, 1)$ and $K_7(4, 2)$ are the 2 cells whose exact value is formalized on both sides; 3 more lower bounds ($K_3(6, 2)$, $K_7(5, 3)$ and $K_7(6, 4)$) carry verified certificates, and 1140 are only claimed. Of the 523 cells whose two bounds now agree, 5 have both bounds certified here at least as witness checked. The lower bounds of Results 8, 11 and 13 are not theorems of the kernel and never enter the ledger as formalized entries. For them version 0.9 adds a separate state, *verified certificate*, for a bound proved by a reduction on paper and certificates

checked by an exact program outside Lean, with the certificates, the checker and the reviews in the provenance; it is computed by the same script and is never counted among the formalized bounds. The new upper bound $K_7(6, 4) \leq 14$ and, from version 0.9.1, the upper bound $K_7(5, 3) \leq 17$ by our own 17-word code are theorems of the kernel; since both codes are also checked by the C verifier, they enter as independently reproduced.

Propagation. A script applies the inequalities between cells that are theorems here (the construction rules above, read forwards for upper bounds and contrapositively for lower bounds) together with the sphere-covering bound, up to a fixed point over all cells of the tables. On the current ledger it finds no inconsistency. With the three new values it finds no consequence outside their own cells: every neighbour already has a better bound than the one implied (for example $K_7(6, 4) \geq 14$ gives $K_8(6, 4) \geq 14$ by projection of the alphabet, against a known 15).

14. METHOD: WITNESS, INDEPENDENT VERIFIER, KERNEL, OBJECTIVE BOUND

Every bound in this note follows the same pipeline, and no step trusts the one before it; only the statements labelled *Theorem*, the upper bounds of Table 1 and the bounds that the ledger marks as formalized reach step 3.

- (1) *Witness.* A search program, a solver or a language model proposes an object: a list of words for an upper bound, or a reduction with certificates (LRAT refutations, integer Farkas multipliers, VeriPB proofs) for a lower bound. Nothing about the proposer is trusted, and an INFEASIBLE answer of a search proves nothing.
- (2) *Independent verifier.* An exact program that shares no code with the proposer checks the object. For codes it is the official verifier of the repository (`tools/verify/verify.c`, C99 with no dependencies): it rejects any digit $\geq q$, any word of the wrong length, any repeated word and a wrong cardinality, marks the radius- R ball of every word in a bitset of q^n bits, and accepts only with zero uncovered points; it also prints the canonical SHA-256 that identifies the code everywhere else. The script `tools/verify/check_all.sh` runs it on every code of the repository, and on the structured description of each code after expansion, and requires the two digests to agree.
- (3) *Kernel.* When the statement is within reach, it becomes a declaration accepted by the Lean kernel with no `sorry`, no `native_decide` and no axiom beyond `propext`, `Classical.choice` and `Quot.sound`; the solvers and scripts that produced the certificate stay outside the trusted base, because only their output enters the kernel.
- (4) *Objective bound.* The certification state of each side of each cell (Section 13) is computed by a script from the Lean sources and the certificates. The counts quoted in this note are generated from that ledger by `tools/paper/numeros.py` into a file of macros that the source includes, and a test fails if the committed file diverges from the ledger or if the text types one of those counts by hand.

The same discipline applies to the prose: novelty is a reading of the literature and is labelled as such, an adversarial review is reported with its gaps, and a negative result is recorded (Section 9) with no change to any bound.

15. INFRASTRUCTURE

The proposers are cheap and plentiful; what the infrastructure guarantees is that their output is checked the same way every time and that the cost of producing it has a ceiling. *Verification.* Continuous integration compiles the official verifier and runs it on every code of the repository, builds the default Lean target, and builds the syndrome certificates (target `CoveringSyn`) whenever they change; the targets `CoveringHeavy` and `CoveringK742Sat` are too costly for it, are built on demand, and their per-module record (time, memory, exit code) is kept in the repository. *Budgeted batch execution.* Heavy jobs (a `lake build`, a large verification, or a script from an allowlist in the

repository) run on ephemeral spot virtual machines created by a batch executor behind a research server with a per-person ceiling in dollars enforced in code. A request first returns, without running, the hourly rate of the machine, the maximum cost (machines $\times$ hours $\times$ rate) and whether it fits the remaining budget; on confirmation the whole batch is reserved in the ledger of the server before the first machine starts, an open reservation already counts as spent (so concurrent requests cannot exceed the ceiling), and if any machine fails to start the others are deleted and the reservation is returned. Each machine has a maximum run duration of the requested hours plus ten minutes, after which the cloud provider deletes it; machines that finished or overran are deleted when the job is polled, and orphans are swept by label. A script runs only from a commit that is already on the main branch, which the machine checks with `git merge-base --is-ancestor`, receives its shard index and the number of shards, and has no credentials. The same server reserves the worst case before a call to a language model and charges the measured tokens; its answers are hypotheses and enter the pipeline only as witnesses of step 1. This executor (version 2, batches of machines in parallel) is in the main branch of the repository since 7 October 2026.

16. HOW THE CODES WERE FOUND

The first codes came from the generator `lincov` of Marosi’s public repository `Mapika/coldcase` [13] followed by greedy patching. The improvements came from treating the base as the main variable. For $K_7(9, 4)$ we enumerated all 6362 classes of non-degenerate $[9, 3]_7$ codes up to equivalence (the list is complete: a mass-formula check, $\sum_S |\text{PGL}(3, 7)|/|\text{Stab}(S)| = 31\,931\,793\,642$, equals the number of 9-point multisets of $\text{PG}(2, 7)$ that contain a frame) and computed, for the classes with the smallest uncovered set, the exact minimum number of orphan syndromes left by three cosets. The best, 6 orphans against 27 for the original base, gives the 1141-word code after a simulated-annealing search on the residual points, and the 1137-word code after a large-neighbourhood search whose subproblems are solved exactly as integer programs. A ball of radius 4 contains at most 24 of the 2058 residual points, so this base needs at least $\lceil 2058/24 \rceil = 86$ patch words; we use 108, and the gap is open. An exact computation of the minimum over all 6362 classes, and over the 1375 classes of degenerate $[9, 3]_7$ codes, is under audit and is not a claim of this note. For the 1134-word code we restricted the patch to be invariant under translation along a line of C_0 (orbits of 7 words); an integer program whose variables are orbits of candidate patch words, solved to optimality by HiGHS for the two best lines (3.4 and 3.6 hours), shows that 105 words (15 orbits) is the minimum under that symmetry, so going lower needs a less symmetric patch or another base. The three new cells use the same scheme: $K_7(10, 4) \leq 5616$ is 16 cosets of a $[10, 3]_7$ code (5488 words, 3 orphan syndromes, 1029 points) plus 128 words, and the 5607-word code keeps the base with a patch of 119 words, found by a randomized greedy for unicost set cover over the 1029 residual points; $K_5(11, 4) \leq 2875$ is 23 cosets of an $[11, 3]_5$ code with no patch; $K_5(10, 5) \leq 162$ is one $[10, 3]_5$ code (125 words, 8 orphan syndromes, 1000 points) plus 37 words, where a linear-programming bound shows that this base needs at least 22 patch words. For $K_7(8, 3)$ the base of the 1893-word code is five cosets of an $[8, 3]_7$ code (1715 words) with two orphan syndromes (686 points); reoptimizing the patch gives 172 words, which is optimal for that base, since no ball of radius 3 contains more than four of the 686 points and $\lceil 686/4 \rceil = 172$. Every candidate was checked by programs independent of the one that found it before being certified in Lean; the codes, a structured description of each (base, cosets, patch, provenance) and an official checker are in the repository.

17. WHAT THE KERNEL ACCEPTED

On 2 October 2026, on a virtual machine of type `e2-highmem-8`, Lean 4.34.1 and Mathlib v4.34.1: `lake build` (default target, from a fresh clone) finished 8944 jobs in 4 min 16 s, peak 10.0 GB; `lake build CoveringHeavy` (the prefix certificates and $K_2(6, 1) = 12$) finished 9181 jobs in 1 h 54 min, about 9.3 CPU-hours, at most 9.4 GB per process; `lake build CoveringSyn` (the four syndrome

certificates) finished: 8983 jobs in 10 min 35 s on 8 cores (37 CPU-minutes for all four), peak 6.9 GB per process. The fifth syndrome certificate, `CoveringLean.Syn_K1137` ($K_7(9, 4) \leq 1137$), built in 2 min 41 s on 8 cores (331 CPU-seconds), peak 6.7 GB. On 3 October 2026, in a container with 4 cores and 15 GB, the four certificates of version 0.6 built from scratch (their own modules; `Mathlib`, `SynCheck` and `SynBridge` from cache): `Syn_K1134` in 105 s, `Syn_K162` in 101 s, `Syn_K2875` in 364 s and `Syn_K5616` in 654 s of wall-clock time. On 4 October 2026, in the same container, `Syn_K5607` built in 1261 s of wall-clock time. For each of the four, three kinds of mutation were built and rejected by Lean: replacing one codeword, removing one codeword (both for the cardinality and for the coverage check), and corrupting one witness in the coverage table. Also on 4 October 2026, the target `CoveringK742Sat` (Theorem 5) was built on two virtual machines (`t2d-standard-8` and `c2d-highmem-8`): 284 modules, all accepted, 37.8 CPU-hours, 3.7 hours of wall-clock time, peak 9.2 GB per process; the inputs (CNF and trimmed LRAT of each profile) are regenerated deterministically and their SHA-256 are recorded in the repository. The upper bounds $K_7(6, 4) \leq 14$ and $K_7(5, 3) \leq 17$ of Section 8 are in the default target. The computational lower bounds of Sections 7 and 8 were not built by Lean; their measurements are given in those sections. `#print axioms` on Theorems 1, 3, 4, 5, on `CoveringLedger.todas_as_cotas`, on `CoveringK764.K_7_6_4_le_14` and `CoveringK753.K_7_5_3_le_17`, and on every declaration of Table 1 reports `propext`, `Classical.choice` and `Quot.sound` only.

18. LITERATURE SEARCH

For version 0.6 we collected, with a reproducible script, the works citing Kéri’s tables, the book [5] and the main papers on q -ary covering codes, through OpenAlex, zbMATH Open, Crossref, OpenCitations and arXiv, down to two levels of citations (2103 works, 582 open-access PDFs read in full), together with Kéri’s tables and their archived copies. No source we read gives a code smaller than ours in any of the four cells, nor a value of $K_7(4, 2)$. Papers behind a paywall were read only through abstracts and reviews, and a full-text index does not find numbers inside tables, so this is a search, not a proof of novelty. For version 0.9, the interval $15 \leq K_3(6, 2) \leq 17$ was traced to its sources [1, 8]: no later work improves either side, the paper of Bertolo, Östergård and Weakley could only be read through its review, and two 2026 attempts at 16 words close nothing. For $K_7(6, 4)$ and $K_7(5, 3)$ we traced each bound to its source [9, 11, 10]. A novelty check of the three cells on 5 October 2026 (arXiv, OpenAlex citations of the source papers, Zenodo, GitHub repositories of the authors working on these cells, Florath’s post-Kéri compilation and Marosi’s tables [13]) found every cell still at its old values; this is why we call the three values potentially new, that is, not found in the literature we searched, and write “to our knowledge new”. The credit for the previous bounds is: for $K_3(6, 2)$, 15 to Bertolo, Östergård and Weakley [1] (improving 14 of Blass and Litsyn [4]) and 17 to Härmäläinen and Rankinen [8], whose code attains the value; partial exclusions for 16 words were obtained independently by Raval [17] and attempted by Florath [6], without a global bound; for $K_7(6, 4)$, 13 to Haas, Schlage-Puchta and Quistorff [9] and 15 to Kéri and Östergård [11]; for $K_7(5, 3)$, 15 to Haas, Quistorff and Schlage-Puchta and 17 to Rivas Soriano, both as announced in Kéri’s tables [10] (we found no publication of either, so we cite the tables). The published version of [9] and the paper [11] were read only through their preprint or review.

19. RELATED WORK: MATHEMATICS WITH ARTIFICIAL INTELLIGENCE IN 2026

We surveyed, on 7 October 2026, the 2026 announcements of open problems solved with artificial intelligence; the survey, with the primary sources read and what was only seen through secondary sources, is `docs/literatura/IA-2026-problemas-resolvidos.md` in the repository. In the sources we read, none concerns covering codes, the covering radius or the football pool problem. The closest item, on binary codes in the “ten advances” manuscript of OpenAI [15], is

an asymptotic upper bound for *packing* ($A(n, d)$, improving the MRRW bound) and gives no number for a finite cell $K_q(n, R)$. That manuscript is accompanied by Lean 4 proofs whose metadata declare no `sorry` and only the axioms `propext`, `Quot.sound` and `Classical.choice`, and by one “Comparator” challenge per result: a short file that only defines and states, the solution in another module, a list of allowed axioms and an independent kernel checker, so that a reviewer reads only the statement. AlphaProof Nexus [3] couples language-model agents with the Lean compiler and formalizes 9 of 353 Erdős problems end to end, at a cost of some hundreds of dollars per problem; AlphaEvolve [2] lets a language model evolve *programs* that search for constructions, judged by an automatic evaluator, and formalized one of its results in Lean. The repository `openai/math` [16], released on 6 October 2026 with about 722 problems, was scanned by the maintainer, who found nothing on covering codes, $K_q(n, R)$ or Kéri’s tables; we did not read it ourselves and record this as his reading. Among the 2026 work on covering codes itself, none declares the use of artificial intelligence: Marosi [13] (upper bounds by local search and lower bounds by semidefinite programming with rational duals checked exactly), Gijswijt and Polak [7] and Florath [6] are cited throughout this note.

Our position relative to this work is narrow and, we think, complementary. We do not claim that a model solved anything: search programs (and, in the campaign below, language models) are proposers in step 1 of Section 14. What we add is that every bound is published as a certificate anyone can check, with a stated trusted base: a list of words that a dependency-free C program verifies in seconds, a declaration that the Lean kernel accepts with the three standard axioms, or a reduction with LRAT or Farkas certificates and the exact checker that accepts them, together with a ledger of all 1145 cells whose states are computed and never typed. The division of labour of AlphaEvolve (an expensive, creative proposer and a cheap, exact evaluator that decides) and the Comparator format (statements reviewed by people, proofs checked by machines) are the two ideas from that work that fit our setting; the first is being tried in the campaign below, and the second is not yet applied to our certificates.

20. RESULTS OF THE CAMPAIGN OF 7 OCTOBER 2026

This section is pending: it will report, with the same states as the rest of the note, only what passed the official verifier and, for theorems, the kernel.

New upper bounds. *Pending.*

Exact values. *Pending.*

Batch certification in Lean. *Pending.*

Programs evolved by a language model. *Pending.*

Cross-check with `openai/math`. *Pending.*

21. WHAT IS NOT A THEOREM

Novelty is a reading of the literature, not a checked fact. No lower bound of Section 7 or Section 8 is a theorem of the kernel; of their upper bounds, $K_3(6, 2) \leq 17$, $K_7(6, 4) \leq 14$ and $K_7(5, 3) \leq 17$ are. For $K_3(6, 2) = 17$ the trusted base is the completeness proof of Lemma 7, written on paper and reviewed adversarially; the enumeration and normalization programs; and the exact Farkas checkers, which rebuild every system from its description. The linear-programming solver is not trusted. The independent reproduction has its own reduction, enumeration and checkers (VeriPB, `lrat-check`, an exact Farkas checker), so a single error in either pipeline would not by itself invalidate the result. For $K_7(6, 4) = 14$ and $K_7(5, 3) = 17$ the trusted base is Lemmas 9 and 10 and the symmetry-breaking steps (a)–(g), written on paper (Lemma 10 after a correction); the CNF encoder; the cube splitting for two profiles of $K_7(5, 3)$, whose completeness was checked by SAT with checked proofs; and `lrat-check`. The SAT solvers are not trusted. Of the 201 376 profiles and 6765 cubes of $K_7(5, 3)$ with 16 words, only a sample was checked by a second checker; 9 large

proofs of the review’s sample were not regenerated. These results also inherit lower bounds from the literature: $K_7(5, 3) \geq 14$ for $K_7(6, 4)$; $K_6(4, 2) = 15$ and $K_7(4, 2) \geq 17$ for $K_7(5, 3) \geq 16$, and $K_7(4, 2) \geq 17$ for $K_7(5, 3) \geq 17$ (the last one is also Theorem 5). The autopsy of the hard instances at the end of Section 7 is an observation, not part of any proof. The search programs are not verified; only their output is. For $K_7(4, 2) \geq 19$ the kernel checks the reduction, the bridge to the CNF, the CNF itself and the 70 refutations; the SAT solver, the trimming tool and the Python scripts that split the proofs into modules are not verified, and need not be, since only their output enters the kernel. In the ledger (Section 13) only the upper bounds marked as formalized and the two lower bounds above are theorems; every other bound is a citation of a published table. That the formal statements say what is intended is a human review: the reader should check the statements of Theorems 1, 3, 4, 5 and the definitions of `Covers`, `IsK` and `CoveringUB.K`. Two files in the tree, `A5b_Stress.lean` and `A6d_SearchHeavy.lean`, are not imported and are not claimed.

REFERENCES

- [1] R. Bertolo, P. R. J. Östergård and W. D. Weakley, *An updated table of binary/ternary mixed covering codes*, J. Combin. Des. **12**(3) (2004), 157–176, doi:10.1002/jcd.20008.
- [2] Google DeepMind, with B. Georgiev, J. Gómez-Serrano, T. Tao and A. Z. Wagner, *AlphaEvolve* applied to 67 mathematical problems, arXiv:2511.02864, 2025.
- [3] Google DeepMind, *AlphaProof Nexus*, arXiv:2605.22763, 2026 (version 2, 8 June 2026).
- [4] U. Blass and S. Litsyn, *Several new lower bounds for football pool systems*, Ars Combin. **50** (1998), 297–302.
- [5] G. Cohen, I. Honkala, S. Litsyn and A. Lobstein, *Covering Codes*, North-Holland, 1997.
- [6] A. Florath, *Formal foundations and proof-carrying certificates for q -ary covering codes in Lean 4*, arXiv:2606.09600, 2026; failure notes for $K_3(6, 2)$ in <https://github.com/florath/covering-codes-lean>, file `docs/failures/K_3_6_2.md`.
- [7] D. Gijswijt and S. Polak, *Semidefinite lower bounds for covering codes*, arXiv:2504.01932, 2025.
- [8] H. O. Härmäläinen and S. Rankinen, *Upper bounds for football pool problems and mixed covering codes*, J. Combin. Theory Ser. A **56** (1991), 84–95, doi:10.1016/0097-3165(91)90024-B.
- [9] W. Haas, J.-C. Schlage-Puchta and J. Quistorff, *Lower bounds on covering codes via partition matrices*, J. Combin. Theory Ser. A **116** (2009), 478–484, doi:10.1016/j.jcta.2008.06.008.
- [10] G. Kéri, *Tables for bounds on covering codes*, <https://old.sztaki.hu/~keri/codes/>, last update 25 November 2011, accessed 5 October 2026.
- [11] G. Kéri and P. R. J. Östergård, *Bounds for covering codes over large alphabets*, Des. Codes Cryptogr. **37**(1) (2005), 45–60, doi:10.1007/s10623-004-3804-8.
- [12] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, CADE 28, LNCS **12699**, Springer, 2021, 625–635.
- [13] M. Marosi, *New upper and lower bounds on covering codes $K_q(n, R)$ for alphabets of size $5 \leq q \leq 21$* , arXiv:2608.19872v3, 2 September 2026 (version 1, August 2026, had $K_7(9, 4) \leq 1743$).
- [14] The Mathlib Community, *Mathlib*, <https://github.com/leanprover-community/mathlib4>, release v4.34.1.
- [15] OpenAI, *Ten advances* (manuscript of 1 August 2026, revised 6 August 2026), <https://cdn.openai.com/pdf/ten-proofs-oai.pdf>; Lean sources at <https://github.com/openai/ten-proofs>.
- [16] OpenAI, *openai/math*, <https://github.com/openai/math>, created 6 October 2026.
- [17] R. R. Raval, *Six certified branch exclusions for the ternary covering problem $K_3(6, 2)$* , Zenodo, doi:10.5281/zenodo.22510341, version 0.1.1, 7 September 2026.
- [18] R. G. Stanton and J. G. Kalbfleisch, *Covering problems for dichotomized matchings*, Aequationes Math. **1** (1968), 94–103.

UNIVERSIDADE DE CAXIAS DO SUL, BRAZIL
 Email address: thiagosleman@gmail.com